SIMATS ENGINEERING

DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 4 – PART II

ALGORITHM ENGINEERING: IMPLEMENT, TEST, DEBUG & OPTIMISE

Course Outcome: CO4 – Solve optimization problems using Dynamic Programming and Greedy strategies
Nature: Individual
Duration: One Week
Total Marks: 50
Environment: Compiler/IDE of student's choice
Prerequisite: Completion of CO4 – Assignment Part I

ALURI POOJITHA(192511230)

TASK 1 – BUILD AND VALIDATE A DYNAMIC PROGRAMMING SOLUTION

Word Wrap as a Document-Layout Engine

8 Marks

A document-processing system must arrange words into lines of maximum width W.

Unused spaces at the end of a line create a penalty. Assume the penalty for a non-final line containing s unused spaces is:

$$Penalty = s^2$$

The final line carries no penalty.

Your objective is to produce the arrangement with a minimum **total penalty**.

1.1 Implement the Solution | 3 Marks

Implement a Dynamic Programming solution.

Your program must output:

- Minimum total penalty
- Words placed on each line
- Unused spaces on each line
- Penalty contributed by each line

Do not output only the minimum cost.

1.2 Expose the DP Behaviour | 2 Marks

Instrument your program to display sufficient intermediate DP states to demonstrate **how the optimum was constructed**.

Explain what each displayed state means.

1.3 Requirement Change | 2 Marks

Run the same text using at least **three different line widths**.

Before executing each modified case, predict:

- Whether the line arrangement will change
- Where you expect the first change to occur
- Whether the total penalty should increase or decrease

Compare your predictions with the actual output.

1.4 Boundary Test | 1 Mark

Design one boundary case that could expose an implementation defect.

Explain why you selected it.

Evidence Expected

Source code, test inputs, outputs, relevant DP states, predictions, modified cases and interpretation.

TASK 2 – GREEDY vs DYNAMIC PROGRAMMING

Build It, Break It, Explain It

9 Marks

A logistics company has a vehicle of limited capacity. Each item has:

Item ID | Weight | Value

Implement:

Solution A

Fractional Knapsack using a Greedy strategy

Solution B

0/1 Knapsack using Dynamic Programming

Your implementation must report the actual items/fractions selected—not merely the optimum value.

2.1 Normal Case | 2 Marks

Create a dataset on which the value-to-weight greedy reasoning produces a strong/optimal result for the applicable formulation.

Show the decision sequence.

2.2 Break the Greedy Intuition | 3 Marks

Construct your own 0/1 Knapsack instance for which applying the same value-to-weight greedy intuition fails to produce the optimum.

Your program must demonstrate:

$$GreedyResult \neq DPOptimum$$

Identify the **first greedy commitment** that prevents the optimum.

2.3 Engineering Comparison | 2 Marks

Instrument both solutions sufficiently to compare:

- Computational work
- Memory requirement
- Solution quality
- Optimality guarantee

Do not rely on execution time alone.

2.4 Changed Requirement | 2 Marks

Management now states:

> "The decision must be returned extremely quickly. A near-optimal result may be acceptable."

Determine how this requirement affects your strategy selection.

Your conclusion must be based on the behaviour of **your implementations**, not a generic textbook statement.

Evidence Expected

Two implementations, selected-item evidence, student-designed counterexample, operation/state evidence and an engineering interpretation.

TASK 3 – ROUTING ENGINE UNDER CHANGING NETWORK CONDITIONS

Dijkstra + Bellman-Ford + Floyd + Warshall

10 Marks

You are developing the routing component of a logistics platform.

Create a weighted network containing at least **8 vertices and 12 edges**.

Stage A – Normal Routing

Implement **Dijkstra's Algorithm**.

For a selected source, display:

- Shortest distance
- Actual path
- Predecessor information
- Order in which vertices become finalised

Stage B – Introduce a Negative Edge

Modify your network by introducing a carefully selected negative-weight edge.

Before executing the modified graph, predict:

- Whether your Dijkstra implementation will remain valid
- Which route/decision may be affected

Now implement/run **Bellman-Ford** and compare the result.

Your test graph must be deliberately designed to expose a meaningful difference where possible; simply inserting an irrelevant negative edge is insufficient.

Stage C – Production Failure

Introduce a **reachable negative-weight cycle**.

Your program must detect and report the condition rather than returning misleading shortest-path results.

Explain how your implementation detects it.

Stage D – Change the Requirement

The analytics team now needs:

1. Shortest-path cost between **every pair of vertices**
2. Whether one location is **reachable** from another

Implement:

- **Floyd's Algorithm**
- **Warshall's Algorithm**

Display the resulting matrices.

Evidence Expected

Graph definition, source code, route reconstruction, prediction before modification, negative-edge investigation, negative-cycle detection, Floyd distance matrix, Warshall reachability matrix and interpretation.

TASK 4 – SMART MANUFACTURING OPTIMISATION

Dynamic Programming for Assembly Line Scheduling

7 Marks

A manufacturing company operates **two parallel assembly lines**. A product must pass through a sequence of processing stations.

At each stage, the product may:

- Continue on the same assembly line, or
- Transfer to the other assembly line before entering the next station.

Each station has a processing time, and switching between assembly lines incurs an additional **transfer time**.

The production engineer needs a system that determines:

The minimum time required to complete one product and the sequence of assembly-line stations that achieves it.

Task 4.1 – Build the Optimisation Engine | 3 Marks

Implement a **Dynamic Programming solution** for the assembly-line scheduling problem.

Your program must accept:

- Processing time at each station on Line 1
- Processing time at each station on Line 2
- Transfer times between the two lines
- Entry times
- Exit times

The program must determine and display:

- Minimum total production time
- Assembly line selected at each station
- Transfer points between lines
- Final optimal route

For example, the output should make the route understandable:

Entry

↓

Line 1 – Station 1

↓

Line 1 – Station 2

↓ Transfer

Line 2 – Station 3

↓

Line 2 – Station 4

↓

Exit

Minimum Production Time = ...

Returning only the minimum numerical value is **not sufficient**.

Task 4.2 – Expose the Decision Process | 1 Mark

Instrument your program to display the important DP decisions at each station.

For each station, show the cost of:

Stay on the current line

versus

Transfer from the other line

and indicate which decision was selected.

Evidence Expected

A trace such as:

Station	Reach Line 1 by Staying	Reach Line 1 by Transfer	Selected	Reach Line 2 by Staying	Reach Line 2 by Transfer	Selected
2	...	...	...	...	...	...
3	...	...	...	...	...	...

The objective is to make the **optimal substructure visible through execution**.

Task 4.3 – Predict Before You Execute | 1 Mark

Select **one transfer time** and increase it substantially.

Before running the modified program, predict:

- Whether the optimal route will change

- Which transfer decision is most likely to be affected
- Whether the total production time will increase

Record your prediction first.

Then execute the program and compare the observed behaviour with your prediction.

Task 4.4 – Production Disruption Challenge | 1 Mark

A machine at **one station becomes slower because of a temporary production issue**.

Increase the processing time of that station.

Determine:

- Whether the optimal route changes
- Whether the algorithm moves production to the alternative line
- At which station the change in route occurs
- How the minimum production time is affected

Explain **why the new route is optimal**.

Task 4.5 – Engineering Extension | 1 Mark

Management now proposes installing a **third assembly line**.

Do **not** implement the three-line version.

Instead, explain:

> **What would have to change in your DP state, transition logic and route-reconstruction mechanism to support three assembly lines?**

Your answer should demonstrate that you understand the underlying optimisation model rather than only the two-line implementation.

Evidence Expected

Working implementation, reconstructed optimal route, intermediate DP decisions, prediction-before-execution evidence, production-disruption experiment and reasoned extension to three lines.

TASK 5 – THREE GREEDY ENGINEERS, ONE NETWORK

Kruskal + Prim + Borůvka

7 Marks

A company must connect all facilities in a network while minimising total infrastructure cost.

Create/use the same connected weighted graph and implement:

- Kruskal's Algorithm
- Prim's Algorithm
- Borůvka's Algorithm

Do Not Compare Only the Final Cost

Instrument each implementation to display the sequence in which the solution is constructed.

For each algorithm determine:

- What constitutes its next **safe edge**
- What information must be maintained
- How the partial solution grows
- Final MST edges
- Total cost

Stress the Design

Modify selected edge weights to create a situation where **multiple MSTs with the same total cost** are possible.

Determine:

> Can the algorithms return different edge sets while all still be correct?

Explain your observation.

Industry Decision

Consider three deployment environments:

A. Sparse network
B. Dense network
C. Large network processed using parallel/distributed resources

Discuss which characteristics of the three algorithms matter when selecting an implementation for each environment.

Do not provide a selection based only on memorised complexity expressions; connect your reasoning to algorithm behaviour and data structures.

Evidence Expected

Three implementations, edge-selection traces, modified graph, multiple-MST evidence and deployment-oriented analysis.

TASK 6 – LAST-MILE DELIVERY ROUTE OPTIMISATION

Travelling Salesperson Problem Using Dynamic Programming 9 Marks

A logistics company operates from a central distribution hub and must make deliveries to several locations.

A delivery vehicle:

- Starts from the distribution hub,
- Visits **every delivery location exactly once**, and
- Returns to the distribution hub.

The travel cost between every pair of locations is known.

The objective is to:

Determine the delivery route having the minimum total travel cost.

You are required to develop a **Dynamic Programming solution** for the problem.

Task 6.1 – Model the Optimisation Problem | 1 Mark

Clearly identify:

- Input
- Output
- Objective

- Constraints

Explain what information must be retained at an intermediate stage to distinguish two partially completed delivery routes.

Evidence Expected

A concise computational model showing why the **current location alone is insufficient** to represent the state of the problem.

Task 6.2 – Design the DP State | 2 Marks

Define a DP state of the form:

$$DP[S][j]$$

where S represents a set of locations already visited and j represents the current location.

Explain precisely:

- What `DP[S][j]` stores
- What the base state represents
- How a larger state is constructed from smaller states
- How the final return to the distribution hub is incorporated

Formulate the recurrence used by your implementation.

Evidence Expected

State definition, base condition, transition/recurrence and final solution condition.

Task 6.3 – Implement the Route Optimisation Engine | 2 Marks

Implement the Dynamic Programming solution.

Use a dataset containing:

- **1 distribution hub**
- At least **6 delivery locations**
- Travel costs between the locations

Your program must display:

- Minimum total travel cost
- Complete optimal route
- Order in which locations are visited
- Return to the distribution hub

For example:

Hub → B → E → A → F → C → D → Hub

Minimum Travel Cost = ...

Returning only the minimum cost is **not sufficient**.

Evidence Expected

Source code, input cost matrix, optimal route and corresponding minimum cost.

Task 6.4 – Validate the Solution | 1 Mark

For a smaller instance containing the hub and **4 delivery locations**, independently determine the possible route costs or use an exhaustive enumeration to establish the optimum.

Compare this result with the output produced by your DP implementation.

Answer:

> **How does this validation increase your confidence that the DP implementation is correct?**

Evidence Expected

Independent reference result, DP result and comparison.

Task 6.5 – Investigate the State-Space Growth | 1 Mark

Instrument your program to record the number of DP states evaluated.

Run the algorithm for increasing numbers of delivery locations, for example:

$$n = 4, 5, 6, 7, 8, \ldots$$

as far as reasonably supported by your system.

Record:

Number of locations	DP states evaluated	Execution time	Memory observation
4			
5			
6			
7			
8			

Do not rely on execution time alone.

Inquiry

Explain:

> **Why does Dynamic Programming eliminate substantial repeated computation but still not make this problem computationally inexpensive for large n?**

Relate your explanation to the number of possible subsets.

Task 6.6 – Business Requirement Change | 1 Mark

The company now operates **200 delivery locations** and requires a route to be generated within a few seconds.

Management states:

> **“An excellent route is acceptable even if it cannot be guaranteed to be mathematically optimal.”**

Do **not** attempt to run your exact DP solution for 200 locations.

Based on the state-space behaviour observed in Task 6.5:

- Explain why the exact DP solution becomes impractical.
- State what property of the requirement has now changed.
- Propose the **type of algorithmic strategy** you would investigate instead.
- Explain the trade-off the company would be accepting.

You are not required to implement the alternative.

Evidence Expected

A reasoned engineering response connecting computational scalability with the changed business requirement.

Task 6.7 – Technical Reflection | 1 Mark

In not more than **150 words**, answer:

> **“Dynamic Programming is more efficient than exhaustive enumeration for TSP. Why does this not mean that Dynamic Programming is scalable for every optimisation problem?”**

Your explanation should refer to evidence obtained from your own implementation.

MARK DISTRIBUTION

Task	Engineering Competency	Marks
1	DP implementation & state validation	**8**
2	Greedy vs DP implementation & adversarial testing	**9**
3	Graph optimisation & failure handling	**10**
4	Algorithmic debugging & regression testing	**7**
5	Greedy MST implementation & engineering comparison	**7**
6	Unseen optimisation prototype	**9**

TOTAL	**50**

MANDATORY SUBMISSION EVIDENCE

For each applicable task, submit:

Source Code → Test Input → Expected Behaviour → Actual Output → Intermediate Algorithmic Evidence → Boundary/Failure Test → Interpretation

Screenshots containing only a final output such as:

Minimum Cost = 145

will **not** constitute sufficient evidence.

SOFTWARE ENGINEERING EXPECTATIONS

Your implementation will also be examined for:

- Meaningful variable and function names
- Modular design
- Appropriate data structures
- Separation of algorithm logic from input/output
- Handling of boundary conditions
- Clear assumptions
- Reproducible test cases
- Correct handling of invalid or exceptional algorithmic conditions where relevant

A program that produces the expected output but whose logic cannot be explained or validated will not receive full credit.

IMPORTANT RULE

DO NOT STOP WHEN THE PROGRAM RUNS.

A successful compilation establishes only that the program is syntactically acceptable.

You must establish:

Does it produce the correct solution? → When does it fail? → Why does it work? → What happens when the requirement changes?

ENGINEERING WORKFLOW

DESIGN → IMPLEMENT → INSTRUMENT → TEST → BREAK → DEBUG → VALIDATE → DEFEND

Task 1 – Word Wrap as a Document-Layout Engine (8 marks)

1.1 Implementation (3 marks)

Model. Words w[0..n-1] must be placed in order on lines of width W. A line holding words i..j has length L(i,j) = Σ len(w[k]) + (j−i) (single spaces) and must satisfy L ≤ W. Unused spaces s = W − L. Penalty of a non-final line = s^2; the final line costs 0.

DP definition. dp[i] = minimum total penalty for arranging words i..n−1 when a new line starts at word i. Base: dp[n] = 0. Recurrence: dp[i] = min over feasible j ≥ i of (penalty(i,j) + dp[j+1]), where penalty(i,j) = 0 if j = n−1 else $(W - L(i,j))^2$. choice[i] stores the best j so the lines can be rebuilt by following i → choice[i]+1. Complexity O(n·k) where k is the maximum words per line ($O(n^2)$ worst case); memory O(n).

Test input (14 words), W = 24: "Algorithms turn careful thinking into reliable and efficient programs for every kind of problem".

Source code (algorithm logic in word_wrap_dp/build_lines; printing in report):

```
"""Task 1 - Word wrap with DP (penalty = unused_spaces^2, final line free)."""


def line_length(words, i, j):
    """Characters needed for words[i..j] with single spaces between them."""
    return sum(len(w) for w in words[i:j + 1]) + (j - i)


def word_wrap_dp(words, width, trace=None):
    """dp[i] = minimum penalty for arranging words[i:]; choice[i] = last word index on the line starting at i."""
    n = len(words)
    for w in words:
        if len(w) > width:
            raise ValueError(f"word '{w}' ({len(w)}) longer than width {width}")
    INF = float("inf")
    dp = [INF] * (n + 1)
    choice = [-1] * (n + 1)
    dp[n] = 0                               # base: no words left -> no penalty
    for i in range(n - 1, -1, -1):
        candidates = []
        for j in range(i, n):
            length = line_length(words, i, j)
            if length > width:              # <= width is allowed (boundary!)
```

```
            break
        unused = width - length
        line_pen = 0 if j == n - 1 else unused ** 2
        total = line_pen + dp[j + 1]
        candidates.append((j, unused, line_pen, dp[j + 1], total))
        if total < dp[i]:
            dp[i], choice[i] = total, j
    if trace is not None:
        trace.append((i, candidates, dp[i], choice[i]))
return dp, choice

def build_lines(words, choice):
    lines, i, n = [], 0, len(words)
    while i < n:
        j = choice[i]
        lines.append((i, j))
        i = j + 1
    return lines

def report(words, width, show_trace=False):
    trace = [] if show_trace else None
    dp, choice = word_wrap_dp(words, width, trace)
    print(f"--- Width W = {width} ---")
    if show_trace:
        for i, cands, best, ch in trace:
            print(f"dp[{i:2d}] ('{words[i]}' starts line):")
            for (j, un, lp, rest, tot) in cands:
                print(f"     line = words[{i}..{j}]  unused={un:2d}  linePenalty={lp:3d} +
dp[{j+1}]={rest:4} = {tot:4}")
            print(f"     => dp[{i}] = {best}, best line ends at word {ch}")
    total = 0
    n = len(words)
    for k, (i, j) in enumerate(build_lines(words, choice), 1):
        text = " ".join(words[i:j + 1])
        unused = width - len(text)
        pen = 0 if j == n - 1 else unused ** 2
        total += pen
```

```
        print(f"Line {k}: |{text.ljust(width)}| unused={unused:2d} penalty={pen}")
    print(f"Minimum total penalty = {total}  (dp[0] = {dp[0]})\n")
    return dp[0]

if __name__ == "__main__":
    text = "Algorithms turn careful thinking into reliable and efficient programs for every kind
of problem"
    words = text.split()
    print("Words:", list(enumerate(words)), "\n")
    report(words, 24, show_trace=True)
    print("===== REQUIREMENT CHANGE: other widths =====")
    for W in (18, 30, 40):
        report(words, W)
    print("===== BOUNDARY TEST =====")
    report(["abc", "12345678", "xy", "z"], 8, show_trace=True)
    try:
        report(["abc", "123456789"], 8)
    except ValueError as e:
        print("Handled error:", e)
```

Actual output – arrangement, unused spaces and penalty per line (W = 24):

```
Line 1: |Algorithms turn careful | unused= 1 penalty=1
Line 2: |thinking into reliable  | unused= 2 penalty=4
Line 3: |and efficient programs  | unused= 2 penalty=4
Line 4: |for every kind of       | unused= 7 penalty=49
Line 5: |problem                 | unused=17 penalty=0
Minimum total penalty = 58  (dp[0] = 58)
```

Verification of the total: 1 + 4 + 4 + 49 + 0 = 58, which equals dp[0]. The program reports the arrangement, unused spaces per line and penalty per line, not only the minimum cost.

1.2 Exposing the DP behaviour (2 marks)

The program can print, for every i (computed from right to left), every candidate line starting at word i together with its line penalty, the already-solved dp[j+1], and their sum. The smallest sum becomes dp[i]. Selected trace lines (W = 24):

```
dp[ 2] ('careful' starts line):
```

line = words[2..2] unused=17 linePenalty=289 + dp[3]= 57 = 346

line = words[2..3] unused= 8 linePenalty= 64 + dp[4]= 53 = 117

line = words[2..4] unused= 3 linePenalty= 9 + dp[5]=   5 =   14

=> dp[2] = 14, best line ends at word 4

dp[0] ('Algorithms' starts line):

line = words[0..0] unused=14 linePenalty=196 + dp[1]= 62 = 258

line = words[0..1] unused= 9 linePenalty= 81 + dp[2]= 14 =   95

line = words[0..2] unused= 1 linePenalty= 1 + dp[3]= 57 =   58

=> dp[0] = 58, best line ends at word 2

Complete DP table extracted from the run (W = 24):

i	Word	dp[i]	Best line = words i..choice[i]	Meaning of the state
13	problem	0	13..13	Last word alone: it is the final line, so it is free.
12	of	0	12..13	‘of problem’ forms the final line, free.
11	kind	0	11..13	‘kind of problem’ fits ($16 \leq 24$) as final line.
10	every	0	10..13	‘every kind of problem’ (21) fits as final line.
9	for	49	9..12	‘for every kind of’ (17) costs $7^2 = 49$; adding ‘problem’ would exceed 24.
8	programs	1	8..11	‘programs for every kind’ (23) leaves 1 unused → 1, then dp[12]=0.
7	efficient	4	7..9	‘efficient programs for’ (22) → $2^2 = 4$, then dp[10]=0.
6	and	53	6..8	‘and efficient programs’ → 4 + dp[9]=49.
5	reliable	5	5..7	‘reliable and efficient’ → 4 + dp[8]=1.
4	into	53	4..6	‘into reliable and’ → 49 + dp[7]=4.
3	thinking	57	3..5	‘thinking into reliable’ → 4 + dp[6]=53.
2	careful	14	2..4	‘careful thinking into’ → 9 + dp[5]=5.
1	turn	62	1..3	‘turn careful thinking’ → 9 + dp[4]=53.
0	Algorithms	58	0..2	Overall optimum = 1 + dp[3]; this is the answer.

Interpretation. Each dp[i] is an independent subproblem ("the best layout of the remaining suffix"). The DP does not just pick the fullest line: at i = 0 the shortest first line ('Algorithms' alone, 196) and the two-word line (81) both lose to the three-word line (1) because the remaining suffix is then cheaper (dp[3] = 57). Optimal substructure is visible: dp[0] = 1 + dp[3] re-uses the value dp[3] computed earlier instead of recomputing the layout of the rest for every candidate first line.

1.3 Requirement change – three more line widths (2 marks)

Same text, widths 18, 30 and 40 (the baseline is W = 24, penalty 58). Predictions were recorded **before** running each case, then compared with the actual output.

Width	Prediction (before execution)	Actual result	Verdict
18	Arrangement changes. Line 1 cannot keep 'Algorithms turn careful' (23 > 18), so the first change is on line 1. Narrower lines mean smaller unused counts, so I expect the total penalty to DECREASE even though there are more lines.	6 lines. Line 1 = 'Algorithms turn', penalty 9+4+1+0+1 = 15.	Correct: first change on line 1; penalty decreased 58 → 15.
30	Line 1 can still not take 'thinking' (needs 32 > 30), so line 1 keeps the same words but gets 6 more unused spaces (1 → 7); the first change in word placement should appear from line 2. Wider lines leave bigger gaps, so total penalty should INCREASE.	4 lines. Line 1 = 'Algorithms turn careful' (unused 7), line 2 gains 'and'. Penalty 49+16+4+0 = 69.	Correct: penalty increased 58 → 69.
40	Arrangement changes at line 1, which can now hold 'Algorithms turn careful thinking into' (36). Fewer non-final lines (2 instead of 4) should make the total DECREASE.	3 lines; line 1 = 5 words, penalty 9+25+0 = 34.	Correct: penalty decreased 58 → 34.

Actual output for the modified cases:

```
--- Width W = 18 ---
Line 1: |Algorithms turn   | unused= 3 penalty=9
Line 2: |careful thinking  | unused= 2 penalty=4
```

Line 3: |into reliable and | unused= 1 penalty=1

Line 4: |efficient programs| unused= 0 penalty=0

Line 5: |for every kind of | unused= 1 penalty=1

Line 6: |problem | unused=11 penalty=0

Minimum total penalty = 15 (dp[0] = 15)

--- Width W = 30 ---

Line 1: |Algorithms turn careful | unused= 7 penalty=49

Line 2: |thinking into reliable and | unused= 4 penalty=16

Line 3: |efficient programs for every | unused= 2 penalty=4

Line 4: |kind of problem | unused=15 penalty=0

Minimum total penalty = 69 (dp[0] = 69)

--- Width W = 40 ---

Line 1: |Algorithms turn careful thinking into | unused= 3 penalty=9

Line 2: |reliable and efficient programs for | unused= 5 penalty=25

Line 3: |every kind of problem | unused=19 penalty=0

Minimum total penalty = 34 (dp[0] = 34)

Interpretation. The minimum penalty is not monotonic in W (W=18 → 15, 24 → 58, 30 → 69, 40 → 34). A wider line is not automatically better: what matters is how well the word lengths "pack" into the width. The DP finds the best packing for each W independently, which a rule such as "fill every line as much as possible" (the greedy approach) cannot guarantee.

1.4 Boundary test (1 mark)

Case: W = 8, words = [abc, 12345678, xy, z]. The word '12345678' is exactly as long as the line. **Why chosen:** the classic off-by-one defect is writing `length < W` instead of `length <= W`; such an implementation would wrongly reject a word that exactly fills the line (or call the instance infeasible). It also checks the zero-unused-space case and the 'final line is free' rule. **Expected:** 'abc' alone (5 unused → 25), '12345678' alone (0), 'xy z' final line (free) → total 25.

Line 1: |abc | unused= 5 penalty=25

Line 2: |12345678| unused= 0 penalty=0

Line 3: |xy z | unused= 4 penalty=0

Minimum total penalty = 25 (dp[0] = 25)

Handled error: word '123456789' (9) longer than width 8

The actual result matches the expectation (25). A second failure test – a word longer than W (9 > 8) – is rejected with a clear error instead of silently producing an invalid layout.

Task 2 – Greedy vs Dynamic Programming: Build It, Break It, Explain It (9 marks)

Solution A: Fractional Knapsack (greedy by value/weight ratio, items may be split). **Solution B:** 0/1 Knapsack (table DP over items × capacity, with back-tracking to list the chosen items). A third helper, greedy_01, applies the *same ratio intuition* to 0/1 items (take a whole item if it fits, else skip) so that the greedy intuition can be tested against the DP on the 0/1 formulation.

Source code:

```
"""Task 2 - Fractional (greedy) vs 0/1 (DP) knapsack, with operation counters."""
import random

def fractional_greedy(items, capacity):
    """items: (id, weight, value). Returns (value, picks[(id, fraction)], ops, log)."""
    ops = {"comparisons_in_sort": 0, "scan_steps": 0}
    import functools
    def cmp(a, b):
        ops["comparisons_in_sort"] += 1
        ra, rb = a[2] / a[1], b[2] / b[1]
        return -1 if ra > rb else (1 if ra < rb else 0)
    order = sorted(items, key=functools.cmp_to_key(cmp))
    remaining, value, picks, log = capacity, 0.0, [], []
    for (iid, w, v) in order:
        ops["scan_steps"] += 1
        if remaining == 0:
            break
        take = min(1.0, remaining / w)
        value += take * v
        remaining -= take * w
        picks.append((iid, take))
        log.append(f"item {iid}: ratio={v/w:.2f}, take {take:.2f} -> value={value:.2f}, capacity
left={remaining:.2f}")
    return value, picks, ops, log

def greedy_01(items, capacity):
    """Same value/weight intuition but items cannot be split: take whole item if it fits, else
skip."""
    order = sorted(items, key=lambda it: it[2] / it[1], reverse=True)
    remaining, value, picks, log = capacity, 0, [], []
```

```
    for (iid, w, v) in order:
        if w <= remaining:
            remaining -= w; value += v; picks.append(iid)
            log.append(f"commit item {iid} (ratio {v/w:.2f}): value={value}, capacity
left={remaining}")
        else:
            log.append(f"skip   item {iid} (ratio {v/w:.2f}): weight {w} > capacity left
{remaining}")
    return value, picks, log

def knapsack_dp(items, capacity):
    """Classic table DP. Returns (best, chosen_ids, cell_evaluations, table_cells)."""
    n = len(items)
    table = [[0] * (capacity + 1) for _ in range(n + 1)]
    cells = 0
    for i in range(1, n + 1):
        _, w, v = items[i - 1]
        for c in range(capacity + 1):
            cells += 1
            table[i][c] = table[i - 1][c]
            if w <= c and table[i - 1][c - w] + v > table[i][c]:
                table[i][c] = table[i - 1][c - w] + v
    chosen, c = [], capacity
    for i in range(n, 0, -1):
        if table[i][c] != table[i - 1][c]:
            chosen.append(items[i - 1][0]); c -= items[i - 1][1]
    return table[n][capacity], chosen[::-1], cells, (n + 1) * (capacity + 1)

if __name__ == "__main__":
    print("=== 2.1 NORMAL CASE ===")
    normal = [("A", 10, 60), ("B", 20, 100), ("C", 30, 120)]
    cap = 50
    val, picks, ops, log = fractional_greedy(normal, cap)
    print("Capacity", cap, "Items", normal)
    print("\n".join(log))
    print("Fractional greedy value =", val, "picks =", picks)
    v01, ch, cells, mem = knapsack_dp(normal, cap)
```

```
    print("0/1 DP value =", v01, "items =", ch)
    # case where greedy 0/1 is also optimal
    easy = [("P", 5, 50), ("Q", 4, 36), ("R", 3, 21), ("S", 2, 8)]
    print("\nDataset where even whole-item greedy is optimal, capacity 12:")
    gv, gp, glog = greedy_01(easy, 12); print("\n".join(glog))
    print("greedy_01 value", gv, gp, "| DP", knapsack_dp(easy, 12)[:2])

    print("\n=== 2.2 BREAK THE GREEDY INTUITION (own instance) ===")
    bad = [("X", 6, 30), ("Y", 5, 24), ("Z", 5, 24)]
    cap = 10
    print("Capacity", cap, "Items (id, weight, value):", bad)
    gv, gp, glog = greedy_01(bad, cap)
    print("\n".join(glog))
    print("Greedy 0/1 result :", gv, gp)
    dv, dch, _, _ = knapsack_dp(bad, cap)
    print("DP optimum        :", dv, dch)
    print("Greedy != DP ?", gv != dv, "| gap =", dv - gv, f"({gv/dv:.1%} of optimum)")
    fv, fp, _, _ = fractional_greedy(bad, cap)
    print("Fractional greedy (relaxation, upper bound):", round(fv, 2), [(i, round(f, 2)) for i, f
in fp])

    print("\n=== 2.3 ENGINEERING COMPARISON ===")
    print(f"{'n':>6} {'W':>8} | {'greedy ops':>11} {'DP cells':>12} | {'greedy mem':>10} {'DP
table':>12}")
    random.seed(7)
    for n, W in [(10, 50), (100, 1000), (1000, 10000), (1000, 100000)]:
        its = [(i, random.randint(1, W // 4 + 1), random.randint(1, 100)) for i in range(n)]
        _, _, o, _ = fractional_greedy(its, W)
        _, _, cells, mem = knapsack_dp(its, W) if n * W <= 10**7 else (0, 0, n * (W + 1), (n +
1) * (W + 1))
        print(f"{n:6d} {W:8d} | {o['comparisons_in_sort'] + o['scan_steps']:11d} {cells:12d} |
{n:10d} {mem:12d}")
    print("(last row: DP cell count computed analytically n*(W+1); table not run)")

    print("\n=== 2.4 QUALITY OF whole-item greedy over 2000 random 0/1 instances ===")
    random.seed(1)
    ratios, fails = [], 0
```

```
for _ in range(2000):
    n = random.randint(5, 12); W = random.randint(20, 60)
    its = [(i, random.randint(1, 25), random.randint(1, 60)) for i in range(n)]
    g = greedy_01(its, W)[0]; d = knapsack_dp(its, W)[0]
    if d: ratios.append(g / d)
    fails += g < d
print(f"greedy not optimal in {fails}/2000 = {fails/20:.1f}% of instances")
print(f"average quality = {sum(ratios)/len(ratios):.3f}, worst = {min(ratios):.3f}")
```

2.1 Normal case (2 marks)

Items (ID, weight, value): A(10, 60), B(20, 100), C(30, 120); capacity 50. Ratios: A = 6, B = 5, C = 4.

```
Capacity 50 Items [('A', 10, 60), ('B', 20, 100), ('C', 30, 120)]
item A: ratio=6.00, take 1.00 -> value=60.00, capacity left=40.00
item B: ratio=5.00, take 1.00 -> value=160.00, capacity left=20.00
item C: ratio=4.00, take 0.67 -> value=240.00, capacity left=0.00
Fractional greedy value = 240.0 picks = [('A', 1.0), ('B', 1.0), ('C', 0.6666666666666666)]
0/1 DP value = 220 items = ['B', 'C']
```

Decision sequence: sort by ratio → take all of A (capacity left 40) → take all of B (left 20) → take 20/30 = 0.67 of C (left 0). Value = 240, which is **optimal for the fractional problem** – the ratio-greedy choice is provably safe there because any leftover capacity can always be filled with the next-best ratio. (Note that for the 0/1 version of this same data the optimum is 220, not 160 or 240 – see 2.2.) For a 0/1 dataset where the ratio rule is also optimal:

```
Dataset where even whole-item greedy is optimal, capacity 12:
commit item P (ratio 10.00): value=50, capacity left=7
commit item Q (ratio 9.00): value=86, capacity left=3
commit item R (ratio 7.00): value=107, capacity left=0
skip   item S (ratio 4.00): weight 2 > capacity left 0
greedy_01 value 107 ['P', 'Q', 'R'] | DP (107, ['P', 'Q', 'R'])
```

Here items P, Q, R fill the capacity exactly (5+4+3 = 12), so no capacity is wasted and the greedy result equals the DP result (107).

2.2 Break the greedy intuition (3 marks) – my own counterexample

Capacity 10; items X(w=6, v=30, ratio 5.0), Y(w=5, v=24, ratio 4.8), Z(w=5, v=24, ratio 4.8).

Capacity 10 Items (id, weight, value): [('X', 6, 30), ('Y', 5, 24), ('Z', 5, 24)]
commit item X (ratio 5.00): value=30, capacity left=4
skip item Y (ratio 4.80): weight 5 > capacity left 4
skip item Z (ratio 4.80): weight 5 > capacity left 4
Greedy 0/1 result : 30 ['X']
DP optimum : 48 ['Y', 'Z']
Greedy != DP ? True | gap = 18 (62.5% of optimum)
Fractional greedy (relaxation, upper bound): 49.2 [('X', 1.0), ('Y', 0.8)]

Greedy result = 30 ≠ DP optimum = 48 (greedy reaches only 62.5 % of the optimum). **First greedy commitment that prevents the optimum:** taking X (the highest-ratio item). It leaves 4 units, and neither Y nor Z (weight 5) can fit, so 4 units of capacity are wasted forever. The DP considers the combination {Y, Z}, which fills the capacity exactly (5+5 = 10). The fractional relaxation reaches 49.2, which is an upper bound on the 0/1 optimum – the greedy 0/1 answer is therefore guaranteed to lose capacity to 'indivisibility'.

2.3 Engineering comparison (2 marks)

Counters inside the code measure the work: for the greedy, comparisons during sorting + scan steps; for the DP, the number of table cells evaluated. Random instances, fixed seed (last row's DP cell count is computed analytically as n·(W+1) because that run would take too long – an intentional point):

n	W	greedy ops	DP cells	greedy mem	DP table
10	50	34	510	10	561
100	1000	553	100100	100	101101
1000	10000	8684	10001000	1000	10011001
1000	100000	8721	100001000	1000	100101001

(last row: DP cell count computed analytically n*(W+1); table not run)

Criterion	Greedy (fractional)	DP (0/1)
Computational work	≈ n log n comparisons + n scan steps (8,721 operations for n = 1000, independent of W).	n·(W+1) cell updates (100,001,000 for n = 1000, W = 100,000) – grows with the NUMERIC VALUE of W (pseudo-polynomial).

Criterion	Greedy (fractional)	DP (0/1)
Memory	O(n): only the sorted list (n = 1000 → 1000 entries).	(n+1)(W+1) table to be able to reconstruct the items (≈ 100 million cells in the last row). A value-only version needs O(W), but then the item list cannot be recovered.
Solution quality	Optimal for the fractional problem. For 0/1 items the same rule failed to be optimal in 606 of 2000 random instances (30.3 %); average quality 98.1 % of optimum, worst case 43.3 %; in my counterexample 62.5 %.	Always optimal for 0/1 (verified by back-tracking the chosen items).
Optimality guarantee	Yes for fractional (exchange argument); NO for 0/1.	Yes for 0/1 (exhaustive over subproblems by optimal substructure).

greedy not optimal in 606/2000 = 30.3% of instances
average quality = 0.981, worst = 0.433

2.4 Changed requirement – "returned extremely quickly, near-optimal may be acceptable" (2 marks)

Conclusion drawn from my measurements: the requirement changes the strategy choice from DP to the ratio greedy. Evidence: (1) at n = 1000, W = 100,000 the DP needs 100,001,000 table updates and a table of the same size, whereas the greedy needs 8,721 operations – about 11,000× less work – and its cost does not depend on W at all; (2) on 2000 random 0/1 instances the greedy was on average within 1.9 % of the optimum, so 'near-optimal' is usually satisfied; (3) however it is not a guarantee: the worst random instance reached only 43.3 % and my designed counterexample 62.5 %, and these failures occur when a high-ratio item is large relative to the capacity (the first greedy commitment wastes capacity).

Engineering decision: use the greedy by default for the time-critical path, but add safeguards – e.g. also compute the best single item that fits and return the better of the two, and flag instances where the largest item is a big fraction of the capacity so that the exact DP (or an offline run) is used when time permits. The DP remains the right choice when W is small or when a certified optimum is contractually required, because its cost is acceptable only when

n·W stays small.

Boundary / failure tests (capacity 0, all items too heavy, empty list):

```
== Task 2 boundaries ==
capacity 0           : DP (0, []) | greedy_01 (0, []) | fractional (0.0, [])
every item too heavy : DP (0, []) | greedy_01 (0, [])
empty item list      : DP (0, [])
```

Task 3 – Routing Engine under Changing Network Conditions (10 marks)

Network (directed, 8 vertices, 13 edges):

From	To	Cost	From	To	Cost
A	B	4	D	F	6
A	C	2	E	F	3
C	B	1	E	G	7
B	D	5	B	G	9
C	D	8	F	H	1
C	E	10	G	H	5
D	E	2			

Source code: Dijkstra (binary heap; a vertex is final when popped), Bellman-Ford (V−1 passes plus a V-th detection pass), Floyd and Warshall.

```
"""Task 3 - Dijkstra, Bellman-Ford, Floyd, Warshall on a directed logistics network."""
import heapq

INF = float("inf")
VERTS = list("ABCDEFGH")
BASE_EDGES = [  # (from, to, cost)  : 8 vertices, 13 edges
    ("A", "B", 4), ("A", "C", 2), ("C", "B", 1), ("B", "D", 5), ("C", "D", 8),
    ("C", "E", 10), ("D", "E", 2), ("D", "F", 6), ("E", "F", 3), ("E", "G", 7),
    ("B", "G", 9), ("F", "H", 1), ("G", "H", 5),
]

def adjacency(edges):
    adj = {v: [] for v in VERTS}
    for u, v, w in edges:
        adj[u].append((v, w))
    return adj

def path_to(pred, target):
    path = []
    while target is not None:
        path.append(target); target = pred[target]
```

```
    return path[::-1]


def dijkstra(edges, source):
    """Textbook Dijkstra: a vertex is final once popped (valid only for non-negative
weights)."""
    adj = adjacency(edges)
    dist = {v: INF for v in VERTS}; pred = {v: None for v in VERTS}
    dist[source] = 0
    done, order = set(), []
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if u in done:
            continue
        done.add(u); order.append(u)          # u is FINALISED here
        for v, w in adj[u]:
            if v not in done and d + w < dist[v]:
                dist[v] = d + w; pred[v] = u
                heapq.heappush(heap, (dist[v], v))
    return dist, pred, order


def bellman_ford(edges, source):
    """Returns (dist, pred, has_negative_cycle, passes_used, cycle_witness_edge)."""
    dist = {v: INF for v in VERTS}; pred = {v: None for v in VERTS}
    dist[source] = 0
    passes = 0
    for _ in range(len(VERTS) - 1):
        changed = False
        passes += 1
        for u, v, w in edges:
            if dist[u] != INF and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w; pred[v] = u; changed = True
        if not changed:
            break
    for u, v, w in edges:                     # V-th pass: any further improvement => negative cycle
        if dist[u] != INF and dist[u] + w < dist[v]:
            return dist, pred, True, passes, (u, v, w)
```

```
    return dist, pred, False, passes, None

def floyd(edges):
    idx = {v: i for i, v in enumerate(VERTS)}; n = len(VERTS)
    d = [[INF] * n for _ in range(n)]
    for i in range(n): d[i][i] = 0
    for u, v, w in edges: d[idx[u]][idx[v]] = min(d[idx[u]][idx[v]], w)
    for k in range(n):
        for i in range(n):
            for j in range(n):
                if d[i][k] + d[k][j] < d[i][j]:
                    d[i][j] = d[i][k] + d[k][j]
    return d

def warshall(edges):
    idx = {v: i for i, v in enumerate(VERTS)}; n = len(VERTS)
    r = [[i == j for j in range(n)] for i in range(n)]
    for u, v, _ in edges: r[idx[u]][idx[v]] = True
    for k in range(n):
        for i in range(n):
            for j in range(n):
                r[i][j] = r[i][j] or (r[i][k] and r[k][j])
    return r

def show_sp(title, dist, pred, order=None):
    print(title)
    if order: print("Finalisation order:", " -> ".join(order))
    print(f"{'V':2} {'dist':>5} {'pred':>5}  path")
    for v in VERTS:
        d = dist[v]
        print(f"{v:2} {('inf' if d == INF else d):>5} {str(pred[v]):>5}  {' -> '.join(path_to(pred,
v)) if d != INF else '-'}")
    print()

def show_matrix(title, m, fmt):
    print(title); print("     " + " ".join(f"{v:>4}" for v in VERTS))
    for i, v in enumerate(VERTS):
```

```
        print(f"{v:>3}  " + " ".join(f"{fmt(x):>4}" for x in m[i]))
    print()

if __name__ == "__main__":
    print("GRAPH (directed):", BASE_EDGES, "\n")
    d, p, o = dijkstra(BASE_EDGES, "A"); show_sp("=== STAGE A: Dijkstra from A ===", d, p, o)

    NEG = [(u, v, w) for u, v, w in BASE_EDGES] + [("G", "D", -6)]
    print("=== STAGE B: add negative edge G->D = -6 ===")
    d2, p2, o2 = dijkstra(NEG, "A"); show_sp("Dijkstra on modified graph", d2, p2, o2)
    bd, bp, cyc, passes, _ = bellman_ford(NEG, "A"); show_sp(f"Bellman-Ford (passes={passes}, negative cycle={cyc})", bd, bp)
    print("Differences (Dijkstra vs Bellman-Ford):")
    for v in VERTS:
        if d2[v] != bd[v]: print(f"  {v}: Dijkstra={d2[v]}  BF={bd[v]}  (BF path: {' -> '.join(path_to(bp, v))})")

    print("\n=== STAGE C: make G->D = -10  (cycle D->E->G->D = 2+7-10 = -1) ===")
    CYC = [(u, v, w) for u, v, w in BASE_EDGES] + [("G", "D", -10)]
    cd, cp, has, passes, wit = bellman_ford(CYC, "A")
    if has: print(f"NEGATIVE CYCLE DETECTED: edge {wit[0]}->{wit[1]} (w={wit[2]}) still relaxes on pass V={len(VERTS)}. Shortest paths undefined; results suppressed.")
    print("Dijkstra on this graph (misleading):", dijkstra(CYC, "A")[0])

    print("\n=== STAGE D: Floyd & Warshall on the Stage-B graph (negative edge, no negative cycle) ===")
    fm = floyd(NEG); show_matrix("Floyd all-pairs shortest distance", fm, lambda x: "inf" if x == INF else x)
    print("Negative cycle check (any d[i][i] < 0)?", any(fm[i][i] < 0 for i in range(len(VERTS))), "\n")
    wm = warshall(NEG); show_matrix("Warshall reachability (1 = reachable)", wm, lambda x: int(x))
    fmc = floyd(CYC)
    print("Floyd on Stage-C graph, diagonal:", [fmc[i][i] for i in range(len(VERTS))], "-> negative diagonal entries reveal the cycle")
```

Stage A – Normal routing (Dijkstra from A)

=== STAGE A: Dijkstra from A ===

Finalisation order: A -> C -> B -> D -> E -> G -> F -> H

V	dist	pred	path
A	0	None	A
B	3	C	A -> C -> B
C	2	A	A -> C
D	8	B	A -> C -> B -> D
E	10	D	A -> C -> B -> D -> E
F	13	E	A -> C -> B -> D -> E -> F
G	12	B	A -> C -> B -> G
H	14	F	A -> C -> B -> D -> E -> F -> H

Vertices become final in the order A, C, B, D, E, G, F, H. For example, the route to H is A → C → B → D → E → F → H (cost 14), reconstructed by following the predecessor array backwards from H.

Stage B – Introducing a negative edge

Deliberately chosen edge: G → D with cost −6. Prediction (before execution): Dijkstra will NOT remain valid. In Stage A, D is finalised (cost 8) before G (cost 12) is even processed. Dijkstra never reopens a finalised vertex, so it cannot discover the cheaper path A → C → B → G → D = 12 − 6 = 6. The decision affected is the route to D, and consequently every route that goes through D: E, F and H. (A negative edge into a vertex that is finalised late would be irrelevant; this one is placed so that a finalised vertex's distance becomes wrong.) Also no negative cycle is created because D → E → G → D costs 2 + 7 − 6 = 3 > 0.

=== STAGE B: add negative edge G->D = -6 ===

Dijkstra on modified graph

Finalisation order: A -> C -> B -> D -> E -> G -> F -> H

V	dist	pred	path
A	0	None	A
B	3	C	A -> C -> B
C	2	A	A -> C
D	8	B	A -> C -> B -> D
E	10	D	A -> C -> B -> D -> E
F	13	E	A -> C -> B -> D -> E -> F
G	12	B	A -> C -> B -> G
H	14	F	A -> C -> B -> D -> E -> F -> H

Bellman-Ford (passes=3, negative cycle=False)

V dist pred path

A 0 None A

B 3 C A -> C -> B

C 2 A A -> C

D 6 G A -> C -> B -> G -> D

E 8 D A -> C -> B -> G -> D -> E

F 11 E A -> C -> B -> G -> D -> E -> F

G 12 B A -> C -> B -> G

H 12 F A -> C -> B -> G -> D -> E -> F -> H

Differences (Dijkstra vs Bellman-Ford):

D: Dijkstra=8 BF=6 (BF path: A -> C -> B -> G -> D)

E: Dijkstra=10 BF=8 (BF path: A -> C -> B -> G -> D -> E)

F: Dijkstra=13 BF=11 (BF path: A -> C -> B -> G -> D -> E -> F)

H: Dijkstra=14 BF=12 (BF path: A -> C -> B -> G -> D -> E -> F -> H)

Comparison: Dijkstra still reports D = 8, E = 10, F = 13, H = 14 – values that look plausible but are wrong. Bellman-Ford (which re-examines every edge in every pass) finds D = 6, E = 8, F = 11, H = 12. The prediction was correct: the four vertices D, E, F, H are wrong, and the true route to H is A → C → B → G → D → E → F → H, which Dijkstra missed entirely.

Stage C – Reachable negative-weight cycle

Edge G → D is changed to −10, giving the cycle D → E → G → D with cost 2 + 7 − 10 = −1, reachable from A.

=== STAGE C: make G->D = -10 (cycle D->E->G->D = 2+7-10 = -1) ===

NEGATIVE CYCLE DETECTED: edge D->E (w=2) still relaxes on pass V=8. Shortest paths undefined; results suppressed.

How detection works: if no negative cycle exists, every shortest path has at most V − 1 = 7 edges, so after V − 1 passes of relaxing all edges no distance can improve. The implementation performs one extra (V-th) pass; if any edge u → v still satisfies dist[u] + w < dist[v], then a path with ≥ V edges is still getting shorter, which is only possible by going round a negative cycle. The program then reports the condition (with the witness edge) and suppresses the distance table instead of returning misleading numbers. By contrast, Dijkstra

run on the same graph quietly returns ‘normal-looking’ distances (D = 8 …) – exactly the misleading output the task warns against.

Stage D – All-pairs shortest cost (Floyd) and reachability (Warshall)

Both are run on the Stage B graph (negative edge, no negative cycle) to show that Floyd handles negative edges.

=== STAGE D: Floyd & Warshall on the Stage-B graph (negative edge, no negative cycle) ===

Floyd all-pairs shortest distance

	A	B	C	D	E	F	G	H
A	0	3	2	6	8	11	12	12
B	inf	0	inf	3	5	8	9	9
C	inf	1	0	4	6	9	10	10
D	inf	inf	inf	0	2	5	9	6
E	inf	inf	inf	1	0	3	7	4
F	inf	inf	inf	inf	inf	0	inf	1
G	inf	inf	inf	-6	-4	-1	0	0
H	inf	inf	inf	inf	inf	inf	inf	0

Negative cycle check (any d[i][i] < 0)? False

Warshall reachability (1 = reachable)

	A	B	C	D	E	F	G	H
A	1	1	1	1	1	1	1	1
B	0	1	0	1	1	1	1	1
C	0	1	1	1	1	1	1	1
D	0	0	0	1	1	1	1	1
E	0	0	0	1	1	1	1	1
F	0	0	0	0	0	1	0	1
G	0	0	0	1	1	1	1	1
H	0	0	0	0	0	0	0	1

Interpretation. Floyd’s matrix gives the cheapest cost for every pair (e.g. A→H = 12, G→D = −6, G→H = 0 via G→D→E→F→H). ‘inf’ means no route. Warshall’s matrix gives the same ‘inf / finite’ pattern as 0/1: e.g. nothing can be reached from H except itself, and nothing returns to A (column A is 0 except the diagonal). For the Stage C graph, Floyd’s diagonal contains negative values, so negative cycles can also be detected from the matrix:

Floyd on Stage-C graph, diagonal: [0, 0, 0, -1, -1, 0, -2, 0] -> negative diagonal entries reveal the cycle

The cycle vertices D, E and G show negative self-distances (−1, −1, −2). A negative d[i][i] means a closed walk of negative cost exists through vertex i, so shortest paths are undefined there; the exact magnitudes are not meaningful once the cycle is present (the values keep decreasing as the cycle is repeated).

Failure test: negative cycle (Stage C) detected; unreachable vertices keep ‘inf’ (e.g. H → anything); a vertex with no outgoing edges (H) is handled.

Task 4 – Smart Manufacturing Optimisation: Assembly-Line Scheduling (7 marks)

Model. f_l[i] = minimum time for a product to complete station i on line l. Base: f_1[1] = e_1 + a_1[1], f_2[1] = e_2 + a_2[1]. Transition: f_1[i] = a_1[i] + min(f_1[i−1], f_2[i−1] + t_2[i−1]) and f_2[i] = a_2[i] + min(f_2[i−1], f_1[i−1] + t_1[i−1]). Answer: min(f_1[n] + x_1, f_2[n] + x_2). A 'came-from' table stores the line used at station i−1 so the route can be rebuilt by back-tracking from the better exit.

Input used: a1 = [7, 9, 3, 4, 8, 4], a2 = [8, 5, 6, 4, 5, 7], t1 = [2, 3, 1, 3, 4] (Line 1 → 2 after stations 1..5), t2 = [2, 1, 2, 2, 1] (Line 2 → 1), entry e1 = 2, e2 = 4, exit x1 = 3, x2 = 2.

Source code:

```
"""Task 4 - Two-line assembly line scheduling (DP)."""


def assembly_dp(a1, a2, t1, t2, e1, e2, x1, x2):
    """a1/a2: station times; t1[i]: transfer Line1->Line2 after station i; t2[i]: Line2->Line1; e: entry; x: exit.
    Returns total, route[(line, station)], trace rows, transfer points."""
    n = len(a1)
    if n == 0 or len(a2) != n or len(t1) != n - 1 or len(t2) != n - 1:
        raise ValueError("need n>=1 stations and n-1 transfer times per line")
    f = [[0] * n for _ in range(2)]          # f[line][i] = fastest time to finish station i on that line
    came = [[0] * n for _ in range(2)]       # came[line][i] = line used at station i-1
    trace = []
    f[0][0], f[1][0] = e1 + a1[0], e2 + a2[0]
    for i in range(1, n):
        stay1, move1 = f[0][i-1] + a1[i], f[1][i-1] + t2[i-1] + a1[i]
        stay2, move2 = f[1][i-1] + a2[i], f[0][i-1] + t1[i-1] + a2[i]
        f[0][i], came[0][i] = (stay1, 0) if stay1 <= move1 else (move1, 1)
        f[1][i], came[1][i] = (stay2, 1) if stay2 <= move2 else (move2, 0)
        trace.append((i + 1, stay1, move1, "stay" if came[0][i] == 0 else "transfer",
                      stay2, move2, "stay" if came[1][i] == 1 else "transfer"))
    end1, end2 = f[0][n-1] + x1, f[1][n-1] + x2
    total, line = (end1, 0) if end1 <= end2 else (end2, 1)
    route, transfers = [], []
    for i in range(n - 1, -1, -1):
        route.append((line + 1, i + 1))
        prev = came[line][i] if i > 0 else line
        if i > 0 and prev != line:
```

```
            transfers.append((i, prev + 1, line + 1))   # after station i, switch prev->line
        line = prev
    return total, route[::-1], trace, transfers[::-1], (f[0], f[1], end1, end2)

def print_result(title, args):
    total, route, trace, transfers, (f1, f2, end1, end2) = assembly_dp(*args)
    print(f"##### {title}")
    print("f1:", f1, " f2:", f2, f" | with exits: L1={end1}, L2={end2}")
    print(f"{'Stn':>3} | {'L1 stay':>7} {'L1 xfer':>7} {'pick':>8} | {'L2 stay':>7} {'L2 xfer':>7} {'pick':>8}")
    print(f"  1 | entry only: L1 = {f1[0]}, L2 = {f2[0]}")
    for r in trace: print(f"{r[0]:>3} | {r[1]:>7} {r[2]:>7} {r[3]:>8} | {r[4]:>7} {r[5]:>7} {r[6]:>8}")
    print("\nEntry")
    for k, (ln, st) in enumerate(route):
        if k > 0 and route[k-1][0] != ln: print(" | Transfer")
        else: print(" |")
        print(f"Line {ln} - Station {st}")
    print(" |\nExit")
    print(f"Minimum Production Time = {total}")
    print("Transfer points:", [f"after station {s}: Line {a}->Line {b}" for s, a, b in transfers] or "none")
    print()
    return total, route

if __name__ == "__main__":
    a1 = [7, 9, 3, 4, 8, 4]; a2 = [8, 5, 6, 4, 5, 7]
    t1 = [2, 3, 1, 3, 4];    t2 = [2, 1, 2, 2, 1]
    base = (a1, a2, t1, t2, 2, 4, 3, 2)
    print_result("4.1/4.2 BASE CASE", base)

    print("##### 4.3 Transfer time increased: t1 (Line1->Line2 after station 1) 2 -> 20")
    t1m = t1[:]; t1m[0] = 20
    print_result("modified transfer", (a1, a2, t1m, t2, 2, 4, 3, 2))

    print("##### 4.4 Disruption: Line 2 Station 2 (5) becomes 25 (machine slows)")
    a2m = a2[:]; a2m[1] = 25
```

```
print_result("disruption on L2 stn 2", (a1, a2m, t1, t2, 2, 4, 3, 2))
print("##### 4.4b Disruption: Line 1 Station 3 (3) becomes 20")
a1m = a1[:]; a1m[2] = 20
print_result("disruption on L1 stn 3", (a1m, a2, t1, t2, 2, 4, 3, 2))

print("##### Boundary tests")
print_result("single station", ([5], [3], [], [], 1, 1, 1, 1))
try: assembly_dp([1, 2], [1, 2], [], [], 0, 0, 0, 0)
except ValueError as e: print("Handled error:", e)
```

4.1 Optimisation engine (3 marks) and 4.2 Decision trace (1 mark)

```
##### 4.1/4.2 BASE CASE
f1: [9, 18, 20, 24, 32, 35]  f2: [12, 16, 22, 25, 30, 37]  | with exits: L1=38, L2=39
Stn | L1 stay L1 xfer     pick | L2 stay L2 xfer     pick
  1 | entry only: L1 = 9, L2 = 12
  2 |     18     23     stay |     17     16 transfer
  3 |     21     20 transfer |     22     27     stay
  4 |     24     28     stay |     26     25 transfer
  5 |     32     35     stay |     30     32     stay
  6 |     36     35 transfer |     37     43     stay

Entry
 |
Line 1 - Station 1
 | Transfer
Line 2 - Station 2
 | Transfer
Line 1 - Station 3
 | Transfer
Line 2 - Station 4
 |
Line 2 - Station 5
 | Transfer
Line 1 - Station 6
 |
Exit
```

Minimum Production Time = 38

Transfer points: ['after station 1: Line 1->Line 2', 'after station 2: Line 2->Line 1', 'after station 3: Line 1->Line 2', 'after station 5: Line 2->Line 1']

Interpretation of the trace. For station 2, Line 1: staying costs f1[1] + a1[2] = 9 + 9 = 18; transferring costs f2[1] + t2[1] + a1[2] = 12 + 2 + 9 = 23 → stay. Line 2: staying 12 + 5 = 17; transferring 9 + 2 + 5 = 16 → transfer. Each f-value is built only from the two optimal values of the previous station, which is the optimal-substructure property. The final comparison is min(f1[6] + x1, f2[6] + x2) = min(35 + 3, 37 + 2) = min(38, 39) = 38, so the product exits from Line 1 after a transfer at station 6. The route has 4 transfer points.

4.3 Predict before you execute (1 mark)

Change: t1[1] (transfer Line 1 → Line 2 after station 1) increased from 2 to 20. **Prediction recorded first:** (a) The optimal route WILL change, because the base route uses exactly this transfer (Line 1 station 1 → Line 2 station 2). (b) The decision most likely affected is the first one, at station 2 (Line 2 'reach by transfer'). (c) Total time will increase, but by much less than 18 because an alternative (staying on Line 1 for stations 2 and 3) exists and costs only a little more.

4.3 Transfer time increased: t1 (Line1->Line2 after station 1) 2 -> 20

modified transfer

f1: [9, 18, 21, 25, 33, 36] f2: [12, 17, 23, 26, 31, 38] | with exits: L1=39, L2=40

Stn	L1 stay	L1 xfer	pick	L2 stay	L2 xfer	pick
1	entry only: L1 = 9, L2 = 12					
2	18	23	stay	17	34	stay
3	21	21	stay	23	27	stay
4	25	29	stay	27	26	transfer
5	33	36	stay	31	33	stay
6	37	36	transfer	38	44	stay

Entry

|

Line 1 - Station 1

|

Line 1 - Station 2

|

Line 1 - Station 3

| Transfer

```
Line 2 - Station 4
 |
Line 2 - Station 5
 | Transfer
Line 1 - Station 6
 |
Exit
Minimum Production Time = 39
Transfer points: ['after station 3: Line 1->Line 2', 'after station 5: Line 2->Line 1']
```

Observed: route changed – production now stays on Line 1 for stations 1–3, then transfers to Line 2 for stations 4–5, then back to Line 1 for station 6. The L2 transfer option at station 2 became 34, so ‘stay’ (17) was selected. Total time rose only from 38 to 39. All three predictions were confirmed, including the observation that the increase is far smaller than the +18 on the transfer – because the DP routes around the expensive transfer instead of paying for it.

4.4 Production disruption challenge (1 mark)

Experiment: the machine at Line 1, Station 3 (the station used by the base route; time 3) becomes slow: 3 → 20. Result:

```
##### 4.4b Disruption: Line 1 Station 3 (3) becomes 20
##### disruption on L1 stn 3
f1: [9, 18, 37, 28, 36, 36]  f2: [12, 16, 22, 26, 31, 38]  | with exits: L1=39, L2=40
Stn | L1 stay L1 xfer     pick | L2 stay L2 xfer     pick
  1 | entry only: L1 = 9, L2 = 12
  2 |      18      23     stay |      17      16 transfer
  3 |      38      37 transfer |      22      27     stay
  4 |      41      28 transfer |      26      42     stay
  5 |      36      36     stay |      31      36     stay
  6 |      40      36 transfer |      38      47     stay

Entry
 |
Line 1 - Station 1
 | Transfer
Line 2 - Station 2
 |
```

Line 2 - Station 3
|
Line 2 - Station 4
|
Line 2 - Station 5
| Transfer
Line 1 - Station 6
|
Exit
Minimum Production Time = 39
Transfer points: ['after station 1: Line 1->Line 2', 'after station 5: Line 2->Line 1']

Question	**Answer**
Does the optimal route change?	Yes.
Does production move to the alternative line?	Yes: the product is moved to Line 2 and stays there for stations 2–5.
At which station does the route change?	Station 3. Base route: L1-S1, L2-S2, L1-S3, L2-S4, L2-S5, L1-S6. New route: L1-S1, L2-S2, L2-S3, L2-S4, L2-S5, L1-S6. Only station 3 changes lines.
Effect on minimum time?	38 → 39 (+1).

Why the new route is optimal. Passing station 3 on Line 1 would cost 20; Line 2's station 3 costs 6, i.e. +3 compared with the original 3. Moving to Line 2 for station 3 also removes two transfers of the old route (Line 2 → Line 1 before station 3, cost t2[2] = 1, and Line 1 → Line 2 after station 3, cost t1[3] = 1). Net change = +3 − 1 − 1 = +1, and the trace shows that every alternative is at least 39 (e.g. at station 3, Line 1 would cost 37–38, Line 2 only 22). A second experiment (Line 2, station 2: 5 → 25) also changed the route (Line 1 for stations 1–3, then Line 2 for 4–5) with total 39.

Boundary / failure tests: one-station line (route chosen by e + a + x: Line 2, total 5) and an invalid input (wrong number of transfer times) – both handled:

single station
f1: [6] f2: [4] | with exits: L1=7, L2=5
Stn | L1 stay L1 xfer pick | L2 stay L2 xfer pick

1 | entry only: L1 = 6, L2 = 4

Entry
|
Line 2 - Station 1
|
Exit
Minimum Production Time = 5
Transfer points: none

Handled error: need n>=1 stations and n-1 transfer times per line

4.5 Engineering extension – three assembly lines (1 mark)

- **DP state:** the idea of f_l[i] is unchanged ('fastest time to finish station i on line l'), but l now ranges over 3 lines, so each station has 3 values instead of 2 – the table grows from 2×n to 3×n (in general L×n).
- **Transition logic:** staying is still one option, but arriving from another line now has two candidate source lines. f_l[i] = a_l[i] + min(f_l[i−1], min over k ≠ l of (f_k[i−1] + t_{k→l}[i−1])). Transfer times must now be a matrix per station t_{k→l} (6 directed pairs) rather than two arrays; entry times e_l and exit times x_l exist for each line and the final answer is min over l of f_l[n] + x_l. Work per station rises from 4 to 9 evaluations (L^2 in general), so total $O(L^2 \cdot n)$. Because a product cannot 'pass through' an intermediate line without being processed there, the transfer matrix does not need to satisfy the triangle inequality – the DP simply takes the minimum.
- **Route reconstruction:** with two lines a Boolean 'stayed / transferred' flag was enough, because the other line was implicit. With three lines the predecessor table must store the actual source line (1, 2 or 3) for each (line, station). Back-tracking starts from the line with the best f_l[n] + x_l and repeatedly follows came_from[l][i]; a transfer is reported whenever the stored source line differs from the current line.
- **Why it still works:** the optimal substructure is the same – the fastest way to finish a station on line l depends only on the fastest ways to finish the previous station on each line.

Task 5 – Three Greedy Engineers, One Network: Kruskal, Prim, Borůvka (7 marks)

Network (undirected, 8 facilities, 13 edges): A-B 4, A-C 3, B-C 5, B-D 6, C-D 7, C-E 8, D-E 2, D-F 9, E-F 5, E-G 11, F-G 4, F-H 10, G-H 3.

Source code:

```
"""Task 5 - Kruskal, Prim, Boruvka on one network, with construction traces and tie
experiments."""
import heapq

class DSU:
    def __init__(self, items): self.p = {x: x for x in items}
    def find(self, x):
        while self.p[x] != x:
            self.p[x] = self.p[self.p[x]]; x = self.p[x]
        return x
    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb: return False
        self.p[ra] = rb; return True

def kruskal(verts, edges, tie_reverse=False, log=None):
    order = sorted(edges, key=lambda e: (e[2], e[3] if not tie_reverse else -e[3]))
    dsu, mst = DSU(verts), []
    for u, v, w, eid in order:
        ok = dsu.union(u, v)
        if log is not None: log.append(f"  consider {u}-{v} ({w}): {'ACCEPT' if ok else 'reject
(cycle)'}")
        if ok: mst.append((u, v, w, eid))
    return mst

def prim(verts, edges, start, log=None):
    adj = {v: [] for v in verts}
    for u, v, w, eid in edges:
        adj[u].append((w, eid, u, v)); adj[v].append((w, eid, v, u))
    seen, heap, mst = {start}, list(adj[start]), []
    heapq.heapify(heap)
    if log is not None: log.append(f"  start at {start}; tree = {{{start}}}")
```

```
    while heap and len(seen) < len(verts):
        w, eid, u, v = heapq.heappop(heap)
        if v in seen: continue
        seen.add(v); mst.append((u, v, w, eid))
        if log is not None: log.append(f"  cheapest edge leaving tree: {u}-{v} ({w}); tree =
{sorted(seen)}")
        for item in adj[v]:
            if item[3] not in seen: heapq.heappush(heap, item)
    return mst

def boruvka(verts, edges, tie_reverse=False, log=None):
    dsu, mst, rnd = DSU(verts), [], 0
    key = (lambda e: (e[2], -e[3])) if tie_reverse else (lambda e: (e[2], e[3]))   # consistent tie-
break
    while len({dsu.find(v) for v in verts}) > 1:
        rnd += 1
        cheapest = {}
        for e in edges:
            a, b = dsu.find(e[0]), dsu.find(e[1])
            if a == b: continue
            for c in (a, b):
                if c not in cheapest or key(e) < key(cheapest[c]): cheapest[c] = e
        if not cheapest:                    # no edge leaves any component -> graph is disconnected
            break
        if log is not None: log.append(f"  round {rnd}: cheapest edge per component = " +
            ", ".join(f"{c}:{e[0]}-{e[1]}({e[2]})" for c, e in sorted(cheapest.items())))
        for e in set(cheapest.values()):
            if dsu.union(e[0], e[1]): mst.append(e)
        if log is not None: log.append(f"          components now: {len({dsu.find(v) for v in
verts})}")
    return mst

def fmt(mst): return ", ".join(f"{u}-{v}({w})" for u, v, w, _ in sorted(mst, key=lambda e:
(e[2], e[3]))) + f"   TOTAL = {sum(e[2] for e in mst)}"
def edge_set(mst): return frozenset(frozenset((u, v)) for u, v, _, _ in mst)

V = list("ABCDEFGH")
```

```
RAW =
[("A","B",4),("A","C",3),("B","C",5),("B","D",6),("C","D",7),("C","E",8),("D","E",2),
      ("D","F",9),("E","F",5),("E","G",11),("F","G",4),("F","H",10),("G","H",3)]
E = [(u, v, w, i) for i, (u, v, w) in enumerate(RAW)]

if __name__ == "__main__":
   print("GRAPH:", RAW, "\n")
   for name, fn in (("KRUSKAL", lambda l: kruskal(V, E, log=l)), ("PRIM (start A)",
lambda l: prim(V, E, "A", l)), ("BORUVKA", lambda l: boruvka(V, E, log=l))):
       log = []; m = fn(log); print("==", name); print("\n".join(log)); print("  MST:", fmt(m),
"\n")

   print("=== STRESS: modify weights to create ties: A-B 4->3, B-C 5->3, E-F 5->4, B-D
6->7 ===")
   RAW2 =
[("A","B",3),("A","C",3),("B","C",3),("B","D",7),("C","D",7),("C","E",8),("D","E",2),
         ("D","F",9),("E","F",4),("E","G",11),("F","G",4),("F","H",10),("G","H",3)]
   E2 = [(u, v, w, i) for i, (u, v, w) in enumerate(RAW2)]
   results = {}; totals = set()
   for label, m in [("Kruskal (ties by input order)", kruskal(V, E2)), ("Kruskal (ties
reversed)", kruskal(V, E2, True)),
                    ("Prim from A", prim(V, E2, "A")), ("Prim from H", prim(V, E2, "H")),
                    ("Boruvka (tie-break by index)", boruvka(V, E2)), ("Boruvka (tie-break
reversed)", boruvka(V, E2, True))]:
       print(f"{label:32}: {fmt(m)}"); results[label] = edge_set(m); totals.add(sum(e[2] for e
in m))
   print("Distinct edge sets found:", len(set(results.values())), "| distinct totals:",
sorted(totals))
   print("\nBoruvka trace (index tie-break):"); log = []; boruvka(V, E2, False, log);
print("\n".join(log))
   print("\nBoruvka WITHOUT consistent tie-break can pick A-B, B-C and A-C in the same
round -> cycle A-B-C. Using (weight, index) order prevents that.")
```

Construction traces and final MST

```
== KRUSKAL
  consider D-E (2): ACCEPT
  consider A-C (3): ACCEPT
  consider G-H (3): ACCEPT
  consider A-B (4): ACCEPT
  consider F-G (4): ACCEPT
  consider B-C (5): reject (cycle)
  consider E-F (5): ACCEPT
  consider B-D (6): ACCEPT
  consider C-D (7): reject (cycle)
  consider C-E (8): reject (cycle)
  consider D-F (9): reject (cycle)
  consider F-H (10): reject (cycle)
  consider E-G (11): reject (cycle)
  MST: D-E(2), A-C(3), G-H(3), A-B(4), F-G(4), E-F(5), B-D(6)   TOTAL = 27

== PRIM (start A)
  start at A; tree = {A}
  cheapest edge leaving tree: A-C (3); tree = ['A', 'C']
  cheapest edge leaving tree: A-B (4); tree = ['A', 'B', 'C']
  cheapest edge leaving tree: B-D (6); tree = ['A', 'B', 'C', 'D']
  cheapest edge leaving tree: D-E (2); tree = ['A', 'B', 'C', 'D', 'E']
  cheapest edge leaving tree: E-F (5); tree = ['A', 'B', 'C', 'D', 'E', 'F']
  cheapest edge leaving tree: F-G (4); tree = ['A', 'B', 'C', 'D', 'E', 'F', 'G']
  cheapest edge leaving tree: G-H (3); tree = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']
  MST: D-E(2), A-C(3), G-H(3), A-B(4), F-G(4), E-F(5), B-D(6)   TOTAL = 27

== BORUVKA
  round 1: cheapest edge per component = A:A-C(3), B:A-B(4), C:A-C(3), D:D-E(2), E:D-
E(2), F:F-G(4), G:G-H(3), H:G-H(3)
        components now: 3
  round 2: cheapest edge per component = C:B-D(6), E:E-F(5), H:E-F(5)
        components now: 1
```

MST: D-E(2), A-C(3), G-H(3), A-B(4), F-G(4), E-F(5), B-D(6) TOTAL = 27

	Kruskal	**Prim (start A)**	**Borůvka**
Next safe edge	The globally cheapest remaining edge whose endpoints are in different components (it is the lightest edge crossing the cut that separates two components – cut property).	The cheapest edge leaving the current tree (the lightest edge crossing the cut 'tree vs rest').	For EVERY component simultaneously, its cheapest outgoing edge; all of them are safe because each is the lightest edge crossing that component's cut.
Information maintained	Edges sorted by weight + a union-find (disjoint-set) structure to detect cycles.	Set of visited vertices + a priority queue (or key[] array) of candidate edges leaving the tree.	Component id of every vertex (union-find) + the best outgoing edge found so far per component in each round.
How the partial solution grows	A forest: many small trees that merge as edges are accepted (D-E, A-C, G-H, A-B, F-G, then E-F merges two big trees, finally B-D).	One single tree that grows by one vertex at a time (A → C → B → D → E → F → G → H).	All components grow in parallel rounds; here 8 components → 3 after round 1 → 1 after round 2.
Final MST edges	D-E 2, A-C 3, G-H 3, A-B 4, F-G 4, E-F 5, B-D 6	same	same
Total cost	27	27	27

Rejected edges in Kruskal (B-C, C-D, C-E, D-F, F-H, E-G) each close a cycle, which the union-find detects. The three algorithms construct the tree in completely different orders but reach the same MST because all weights are distinct, so the MST is unique.

Stress the design – multiple MSTs

Modified weights: A-B 4→3, B-C 5→3, E-F 5→4 and B-D 6→7 (C-D stays 7). Now the triangle A-B-C has three edges of weight 3, and B-D and C-D both have weight 7.

```
=== STRESS: modify weights to create ties: A-B 4->3, B-C 5->3, E-F 5->4, B-D 6->7 ===
Kruskal (ties by input order)   : D-E(2), A-B(3), A-C(3), G-H(3), E-F(4), F-G(4), B-D(7)
TOTAL = 26
Kruskal (ties reversed)         : D-E(2), A-C(3), B-C(3), G-H(3), E-F(4), F-G(4), C-D(7)
TOTAL = 26
Prim from A                     : D-E(2), A-B(3), A-C(3), G-H(3), E-F(4), F-G(4), B-D(7)
TOTAL = 26
Prim from H                     : E-D(2), B-A(3), A-C(3), H-G(3), F-E(4), G-F(4), D-B(7)
TOTAL = 26
Boruvka (tie-break by index)    : D-E(2), A-B(3), A-C(3), G-H(3), E-F(4), F-G(4), B-D(7)
TOTAL = 26
Boruvka (tie-break reversed)    : D-E(2), A-C(3), B-C(3), G-H(3), E-F(4), F-G(4), C-D(7)
TOTAL = 26
Distinct edge sets found: 2 | distinct totals: [26]

Exhaustive check of all 7-edge subsets: minimum weight = 26, number of distinct MSTs = 6
```

Observation. Yes – correct algorithms can return different edge sets with the same total cost (26). Kruskal with ties broken in input order and Prim and Borůvka (index tie-break) return {A-B, A-C, …, B-D}, while reversing the tie-break gives {A-C, B-C, …, C-D}. An exhaustive check shows exactly 6 minimum spanning trees (any 2 of the 3 triangle edges × either B-D or C-D). All are correct because the cut property only says that a minimum-weight crossing edge is safe – when several tie, any of them is safe – and the cycle property says that in the triangle any one of the equal-heaviest edges may be dropped. Prim from A and from H returned the same edge set here, though in general the start vertex can matter under ties.

Borůvka-specific subtlety (a real implementation defect I found): with equal weights, components may choose different edges of the same weight in the same round and create a cycle (e.g. A-B, B-C and A-C selected by three different components). The implementation

therefore uses a consistent total order (weight, edge index). I also found an infinite loop when the graph is disconnected (no component has an outgoing edge, so the component count never reaches 1); the fix is to stop when no component has a cheapest edge. The boundary test shows all three algorithms now return V−2 = 6 edges for a disconnected graph, which the caller can detect:

disconnected graph, Kruskal: 6 edges returned (a spanning tree needs 7) -> DETECT: graph not connected
disconnected graph, Prim: 6 edges returned (a spanning tree needs 7) -> DETECT: graph not connected
disconnected graph, Boruvka: 6 edges returned (a spanning tree needs 7) -> DETECT: graph not connected

Industry decision

Environment	**Best fit**	**Reasoning based on algorithm behaviour and data structures**
A. Sparse network (E ≈ V)	Kruskal (Prim with a binary heap is a close second)	Kruskal's cost is dominated by sorting E edges and a few near-constant union-find operations; with E close to V the sort is cheap and the edge-list representation is compact. It also works naturally if edges arrive as a stream/file. Prim with a heap only touches edges incident to the tree and is equally good; an adjacency-matrix Prim would waste time on empty cells.
B. Dense network (E ≈ V²)	Prim (array/adjacency-matrix version)	Prim never sorts the E edges: with a key[] array it needs V rounds that each scan V vertices, so it uses only the V² entries it already stores. Kruskal would have to sort ≈ V² edges and Borůvka would rescan all edges every round. Prim keeps one small 'best edge to tree' value per vertex.

Environment	**Best fit**	**Reasoning based on algorithm behaviour and data structures**
C. Very large network, parallel/distributed	Borůvka	Prim is inherently sequential (one tree, one global ‘cheapest edge’ each step) and Kruskal needs a globally sorted edge order and a single shared union-find. In Borůvka every component picks its cheapest outgoing edge independently, so each round is a parallel scan of the edges plus a small reduction (min per component). The number of components at least halves per round, so only O(log V) synchronisation rounds are needed (in my run, 8 components → 3 → 1). The price is a consistent tie-break and communication of component ids each round.

Task 6 – Last-Mile Delivery Route Optimisation: TSP with Dynamic Programming (9 marks)

6.1 Model the optimisation problem (1 mark)

Element	Definition
Input	Hub (city 0) and n delivery locations 1..n; a travel-cost matrix c[i][j] (known for every pair).
Output	A tour Hub → p1 → … → pn → Hub and its total cost.
Objective	Minimise the sum of the costs of consecutive legs of the tour.
Constraints	The tour starts and ends at the hub; every delivery location is visited exactly once.

Why the current location alone is not a state. Two partial routes can both end at location C but have visited different locations, e.g. Hub→A→B→C (visited {A,B,C}) and Hub→D→C (visited {D,C}). The best way to finish depends on which locations are still unvisited, so the two partial routes are not interchangeable. The state must therefore keep (set of visited locations, current location); the order in which the visited ones were visited does not matter for the future, only their cost, so only the cheapest such ordering is kept.

6.2 DP state (2 marks)

- **DP[S][j]** = minimum cost of a path that starts at the hub, visits exactly the delivery locations in S (each once) and ends at j ∈ S.
- **Base state:** S = {j}: DP[{j}][j] = c[hub][j] (go directly from the hub to j).
- **Construction of a larger state:** a state (S, j) is built from smaller states (S \ {j}, k) – the path must have been at some k ∈ S \ {j} just before j.
- **Recurrence:** DP[S][j] = min over k ∈ S, k ≠ j of (DP[S \ {j}][k] + c[k][j]).
- **Final return to the hub:** Optimal cost = min over j of (DP[All][j] + c[j][hub]), where All is the set of all n delivery locations. A parent[S][j] table remembers the best k so the route is rebuilt by back-tracking from the best j.

6.3 Implementation (2 marks)

Subsets are bit-masks (bit k−1 ↔ location k). Dataset: hub + 6 locations A–F:

Source code:

```
"""Task 6 - TSP with Held-Karp DP (hub = city 0)."""
import itertools, random, time, tracemalloc
```

```
def held_karp(cost, names):
    """cost[i][j]; city 0 is the hub. DP[S][j]: cheapest path hub -> (all of S) ending at j, j in S.
    S is a bitmask over delivery locations 1..m (bit k-1 <-> city k)."""
    m = len(cost) - 1
    if m < 1: raise ValueError("need at least one delivery location")
    FULL = (1 << m) - 1
    INF = float("inf")
    dp = [[INF] * (m + 1) for _ in range(1 << m)]
    parent = [[-1] * (m + 1) for _ in range(1 << m)]
    states = transitions = 0
    for j in range(1, m + 1):                    # base: S={j}
        dp[1 << (j - 1)][j] = cost[0][j]; states += 1
    for S in range(1, 1 << m):
        for j in range(1, m + 1):
            if not S & (1 << (j - 1)) or S == (1 << (j - 1)): continue
            prevS = S ^ (1 << (j - 1))
            states += 1
            for k in range(1, m + 1):
                if prevS & (1 << (k - 1)):
                    transitions += 1
                    cand = dp[prevS][k] + cost[k][j]
                    if cand < dp[S][j]: dp[S][j], parent[S][j] = cand, k
    best, last = min((dp[FULL][j] + cost[j][0], j) for j in range(1, m + 1))   # close the tour
    route, S, j = [], FULL, last
    while j != -1:
        route.append(j); pj = parent[S][j]; S ^= 1 << (j - 1); j = pj
    route = [0] + route[::-1] + [0]
    return best, [names[c] for c in route], states, transitions


def brute_force(cost, names):
    m = len(cost) - 1; best = (float("inf"), None); table = []
    for perm in itertools.permutations(range(1, m + 1)):
        path = (0,) + perm + (0,)
        c = sum(cost[a][b] for a, b in zip(path, path[1:])); table.append((c, [names[x] for x in
path]))
        best = min(best, (c, path))
```

```
    return best[0], [names[x] for x in best[1]], table

def random_matrix(n, seed):
    rnd = random.Random(seed)
    c = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        for j in range(i + 1, n + 1): c[i][j] = c[j][i] = rnd.randint(5, 60)
    return c

if __name__ == "__main__":
    names = ["Hub", "A", "B", "C", "D", "E", "F"]
    cost = [[0,12,10,19,8,14,17],
            [12,0,3,7,2,9,11],
            [10,3,0,6,20,5,13],
            [19,7,6,0,4,16,8],
            [8,2,20,4,0,15,9],
            [14,9,5,16,15,0,6],
            [17,11,13,8,9,6,0]]
    print("Cost matrix (Hub + 6 locations):")
    print("      " + " ".join(f"{n:>4}" for n in names))
    for n, row in zip(names, cost): print(f"{n:>5} " + " ".join(f"{x:>4}" for x in row))
    best, route, states, trans = held_karp(cost, names)
    print("\nOptimal route:", " -> ".join(route)); print("Minimum Travel Cost =", best, f"| DP
states = {states}, transitions = {trans}")

    print("\n=== 6.4 VALIDATION: Hub + 4 locations ===")
    c4 = [row[:5] for row in cost[:5]]; n4 = names[:5]
    bf, bfroute, table = brute_force(c4, n4)
    for c, p in sorted(table)[:5]: print(f"  {c:3d}  {' -> '.join(p)}")
    print(f"  ... ({len(table)} permutations, 4! = 24)")
    dpc, dproute, _, _ = held_karp(c4, n4)
    print("Brute force :", bf, " ".join(bfroute)); print("DP          :", dpc, " ".join(dproute));
print("Match:", bf == dpc)
    ok = all(held_karp(random_matrix(k, s), [str(i) for i in range(k+1)])[0] ==
brute_force(random_matrix(k, s), [str(i) for i in range(k+1)])[0] for k in range(2, 8) for s in
range(10))
    print("Extra check - 60 random instances (n=2..7) DP == brute force:", ok)
```

```
    print("\n=== 6.5 STATE-SPACE GROWTH ===")
    print(f"{'n':>3} {'DP states':>10} {'n*2^(n-1)':>10} {'transitions':>12} {'time (s)':>9}
{'peak mem (MB)':>13}")
    for n in range(4, 16):
        mat = random_matrix(n, 100 + n); nm = [str(i) for i in range(n + 1)]
        tracemalloc.start(); t = time.perf_counter()
        _, _, st, tr = held_karp(mat, nm)
        el = time.perf_counter() - t; peak = tracemalloc.get_traced_memory()[1] / 1e6;
tracemalloc.stop()
        print(f"{n:>3} {st:>10} {n * 2 ** (n - 1):>10} {tr:>12} {el:>9.3f} {peak:>13.1f}")
    print("\nn = 200 : n*2^(n-1) =", f"{200 * 2 ** 199:.3e}", "states   (2^200 =",
f"{2**200:.3e})")
    print("Brute-force permutations n! for n=15 :", f"{__import__('math').factorial(15):.3e}",
"| DP states for n=15 :", 15 * 2 ** 14)
```

```
Cost matrix (Hub + 6 locations):
      Hub    A    B    C    D    E    F
  Hub    0   12   10   19    8   14   17
    A   12    0    3    7    2    9   11
    B   10    3    0    6   20    5   13
    C   19    7    6    0    4   16    8
    D    8    2   20    4    0   15    9
    E   14    9    5   16   15    0    6
    F   17   11   13    8    9    6    0

Optimal route: Hub -> D -> C -> F -> E -> B -> A -> Hub
Minimum Travel Cost = 46 | DP states = 192, transitions = 480
```

Optimal route: Hub → D → C → F → E → B → A → Hub, Minimum Travel Cost = 46
(8 + 4 + 8 + 6 + 5 + 3 + 12 = 46; the individual legs add up as shown).

6.4 Validation (1 mark)

```
=== 6.4 VALIDATION: Hub + 4 locations ===
   32  Hub -> B -> A -> C -> D -> Hub
   32  Hub -> D -> C -> A -> B -> Hub
   33  Hub -> A -> B -> C -> D -> Hub
```

33 Hub -> B -> C -> A -> D -> Hub

33 Hub -> D -> A -> C -> B -> Hub

... (24 permutations, 4! = 24)

Brute force : 32 Hub B A C D Hub

DP : 32 Hub D C A B Hub

Match: True

Extra check - 60 random instances (n=2..7) DP == brute force: True

How this increases confidence. Brute-force enumeration over all 4! = 24 orderings is an independent method (no shared logic with the DP), and it gives the same minimum cost (32). The two routes reported differ (Hub-B-A-C-D-Hub vs the reverse) only because the matrix is symmetric, so a route and its reverse cost the same – both are optimal. In addition, the 60 random instances (n = 2…7) matched the brute-force cost, which tests the base states, the transitions and the closing leg on many different matrices. A pass does not prove correctness for large n, but it rules out the typical defects (wrong base case, forgetting the return leg, wrong subset update).

6.5 State-space growth (1 mark)

=== 6.5 STATE-SPACE GROWTH ===

n	DP states	n*2^(n-1)	transitions	time (s)	peak mem (MB)
4	32	32	48	0.000	0.0
5	80	80	160	0.001	0.0
6	192	192	480	0.002	0.0
7	448	448	1344	0.006	0.0
8	1024	1024	3584	0.015	0.1
9	2304	2304	9216	0.042	0.1
10	5120	5120	23040	0.067	0.3
11	11264	11264	56320	0.167	0.7
12	24576	24576	135168	0.651	1.4
13	53248	53248	319488	1.139	2.9
14	114688	114688	745472	2.731	6.0
15	245760	245760	1720320	6.087	12.6

Locations n	**DP states evaluated**	**Execution time (see table above)**	**Memory observation**
4	32	≈ 0 s	< 0.1 MB

Locations n	DP states evaluated	Execution time (see table above)	Memory observation
8	1,024	≈ 0.01 s	≈ 0.1 MB
12	24,576	≈ 0.4 s	≈ 1.4 MB
15	245,760	≈ 6 s	≈ 12.6 MB

(The complete n = 4…15 data is in the output table above: states follow exactly n·2^(n−1); times and memory are machine-dependent, but the doubling trend is not.)

Inquiry – why DP removes repeated work but is still expensive. Exhaustive search examines n! orderings (1.3×10^{12} for n = 15) and recomputes the cost of the same partial route over and over. The DP computes the cost of each (subset, last location) pair once and re-uses it: only 245,760 states for n = 15. But the number of distinct subsets of n locations is 2^n, and each state is a pair (subset, end point), giving n·2^(n−1) states. Every added location multiplies the number of states by about 2 (from 114,688 at n = 14 to 245,760 at n = 15, ratio 2.14) and the work and memory follow. DP replaced the factorial by an exponential – a huge improvement – but 2^n is still exponential.

6.6 Business requirement change – 200 locations in a few seconds (1 mark)

- **Why exact DP is impractical:** from 6.5, states = n·2^(n−1). For n = 200 this is about 1.6×10^{62} states, and each needs its own memory cell and several operations – impossible in time and in memory (my own run already needs ≈ 12 MB and ≈ 6 s at n = 15, and both double with every added location).
- **Property of the requirement that changed:** the requirement moved from ‘must be provably optimal’ to ‘must be fast and good enough’ – the guarantee of optimality is relaxed and the running time (a few seconds) becomes a hard constraint.
- **Strategy to investigate instead:** heuristic / approximate methods, e.g. a constructive heuristic (nearest neighbour or greedy edge / cheapest insertion) to get a starting tour, followed by local-search improvement (2-opt, Or-opt, Lin–Kernighan), possibly with metaheuristics (simulated annealing, genetic algorithm) given a time limit; or an approximation algorithm (MST-based or Christofides for metric costs) if a bounded error is needed.
- **Trade-off accepted:** the company gives up the guarantee of the minimum cost and also the ability to know how far from optimal the route is (unless a lower bound or approximation ratio is used). In exchange it obtains routes in seconds that are usually close to optimal; randomised methods may also return different routes on different runs.

6.7 Technical reflection (≤ 150 words, 1 mark)

Dynamic Programming beats enumeration by solving each subproblem once: for n = 15 my program evaluated 245,760 states instead of about 1.3×10^{12} permutations. However, scalability depends on how many distinct subproblems exist. TSP's subproblems are (subset, end location) pairs, so there are n·2^(n−1) of them; my measurements showed states, time and memory roughly doubling with each added location (114,688 states at n = 14, 245,760 at n = 15). DP therefore turns factorial growth into exponential growth, not into polynomial growth, and at n = 200 it would need about 10^{62} states. DP is scalable only when the number of subproblems is polynomial, as in my assembly-line DP (2n states) – a property of the problem, not of the technique.

Word count: 122

Boundary / failure tests for Task 6:

```
== Task 6 boundaries ==
1 delivery location : (14, ['Hub', 'A', 'Hub'])
2 locations         : (7, ['Hub', 'B', 'A', 'Hub'])
0 locations -> handled error: need at least one delivery location
```

Summary of Evidence and Engineering Reflection

Task	Tests that went beyond ‘it runs’	Failure found / design insight
1 Word wrap	Predictions for 3 widths; exact-fit boundary; word longer than W	Penalty is not monotonic in W; ‘fill each line’ greed is not optimal.
2 Greedy vs DP	Own counterexample (30 vs 48); 2000 random instances; operation counts; boundary cases	Greedy quality 98.1 % on average but only 43.3 % in the worst case; DP cost depends on W.
3 Routing	Negative edge designed to hit a finalised vertex; negative cycle detection; Floyd diagonal check	Dijkstra silently returns wrong results with negative edges; Bellman-Ford’s V-th pass detects cycles.
4 Assembly line	Transfer-time and machine-slowdown experiments; invalid-input check	Route changes could cost only +1 time unit because the DP routes around the change.
5 MST	Three construction traces; tie experiment; exhaustive MST count (6)	Found and fixed a Borůvka infinite loop on disconnected graphs; ties need a consistent order.
6 TSP	Brute-force validation (+60 random cases); state-space measurements n = 4…15	Exact DP is exponential: $\approx 10^{62}$ states at n = 200.